



Clustering experience replay for the effective exploitation in reinforcement learning

Min Li, Tianyi Huang, William Zhu*

Institute of Fundamental and Frontier Sciences, University of Electronic Science and Technology of China, Chengdu 610054, China

ARTICLE INFO

Article history:

Received 7 May 2021

Revised 18 May 2022

Accepted 26 June 2022

Available online 27 June 2022

Keywords:

Reinforcement learning

Clustering

Experience replay

Exploitation efficiency

Time division

ABSTRACT

Reinforcement learning is a useful tool for training an agent to effectively achieve the desired goal in the sequential decision-making problem. It trains the agent to make decision by exploiting the experience in the transitions resulting from the different decisions. To exploit this experience, most reinforcement learning methods replay the explored transitions by uniform sampling. But in this way, it is easy to ignore the last explored transitions. Another way to exploit this experience defines the priority of each transition by the estimation error in training and then replays the transitions according to their priorities. But it only updates the priorities of the transitions replayed at the current training time step, thus the transitions with low priorities will be ignored. In this paper, we propose a clustering experience replay, called CER, to effectively exploit the experience hidden in all explored transitions in the current training. CER clusters and replays the transitions by a divide-and-conquer framework based on time division as follows. Firstly, it divides the whole training process into several periods. Secondly, at the end of each period, it uses k -means to cluster the transitions explored in this period. Finally, it constructs a conditional probability density function to ensure that all kinds of transitions will be sufficiently replayed in the current training. We construct a new method, TD3_CER, to implement our clustering experience replay on TD3. Through the theoretical analysis and experiments, we illustrate that our TD3_CER is more effective than the existing reinforcement learning methods. The source code can be downloaded from <https://github.com/grcai/CER-Master>.

© 2022 Elsevier Ltd. All rights reserved.

1. Introduction

Sequential decision-making is a fundamental issue faced by an agent which is interacting with an environment to achieve the desired goal. It widely exists in real-world applications, such as robot control [1,2] and autonomous driving [3,4]. As a significant branch of machine learning [5], reinforcement learning (RL) [6,7] is a useful tool for this issue. In order to maximize the cumulative reward from the desired goal, RL trains the agent to make decisions by exploiting the experience of this agent transiting from one state to another in the interaction [8]. A lot of useful information, such as the expected cumulative reward, can be mined in the transitions of the agent. Therefore, it is important to effectively exploit the experience in these transitions [9].

Experience replay (ER) [10–12] is used to exploit this experience in many RL methods, such as DQN [13], DDPG [14], TD3 [15], and their variants [16]. For each training time step, ER stores the transition explored at this step into a replay buffer and then re-

plays the transitions in the replay buffer by uniform sampling. However, the last explored transitions are easy to be ignored in this way. Therefore the exploitation efficiency is limited by ER.

As an important improvement of ER, prioritized experience replay (PER) [9] assumes that a transition with a large estimation error of its expected cumulative reward should be frequently learned so as to reduce this error, which is called temporal-difference (TD) error [17]. PER uses the TD error to define the priority of each transition and then replays this transition according to its priority. Especially, it gives the largest priority to the last explored transition before computing its TD error. Thus each transition can be replayed at least once. The TD error of every transition is changed at each training time step, but PER only updates the priorities of the transitions replayed at the current training time step. In this way, the transitions with low priorities cannot be updated, leading to the problem that these transitions are ignored. Thus the exploitation efficiency is also limited by PER.

In this paper, we propose a clustering experience replay (CER) to improve the exploitation efficiency by combining the clustering method [18,19] and the experience replay method into a divide-and-conquer framework. The main idea of our CER is that (1) In

* Corresponding author.

E-mail address: wfzhu@uestc.edu.cn (W. Zhu).

the current training, we should sufficiently mine the experience hidden in all explored transitions. (2) Instead of a time consuming way, replaying all explored transitions, we can cluster them and then replay the sampled ones from all the clusters. CER effectively clusters and replays the transitions by a divide-and-conquer framework based on time division as follows. Firstly, it divides the whole training process into several periods. Secondly, at the end of each period, it uses an efficient clustering method, k -means [20–22], to cluster the transitions explored in this period. Finally, it constructs a conditional probability density function to replay the different kinds of transitions. By this function, CER can sufficiently replay all kinds of transitions in current training to ensure that any kind of transitions will not be ignored. Therefore, our CER can effectively improve the exploitation efficiency in RL.

Our CER can be added to any RL method which needs to replay the transitions. As an implementation of our CER, a new method, TD3_CER, is constructed by adding CER to TD3 [15]. We theoretically analyze that our CER can sufficiently replay all kinds of transitions. In experiments, we compare TD3_CER with TD3 and TD3_PER, which replay the transitions by ER and PER, respectively. The performance of TD3_CER is better than the performances of TD3 and TD3_PER. This result illustrates that the exploitation efficiency of CER is better than that of ER and PER.

The remainder of this paper is organized as follows. In Section 2, the related background knowledge is reviewed. In Section 3, we analyze the limited exploitation efficiency in existing experience replay methods. In Section 4, we introduce our CER and a new method, TD3_CER. We also theoretically analyze CER in this section. Experiments are reported in Section 5 to demonstrate the effectiveness of our TD3_CER. The conclusion is given in Section 6.

2. Background

In this section, firstly, we introduce the formulation of RL. Then we present TD3, an efficient RL method with ER. Finally, as an important improvement of ER, PER is reviewed.

2.1. Reinforcement learning formulation

In RL, an agent interacts with its environment to find the optimal policy by maximizing a numerical reward signal [23,24]. For each discrete time step t , the agent at the state $s_t \in \mathcal{S}$ chooses an action $a_t \in \mathcal{A}$ according to its policy $\pi: \mathcal{S} \rightarrow \mathcal{A}$. Then the agent obtains a reward $r_t = r(s_t, a_t)$ and the successor state s_{t+1} . The transition explored in this time step is denoted as (s_t, a_t, r_t, s_{t+1}) . The discounted cumulative reward from time step t of an episode is defined as $R_t = \sum_{i=t}^{\infty} \gamma^i r(s_i, a_i)$, where $\gamma \in [0, 1)$ is a discount rate [6]. The objective of RL is to maximize $J(\pi) = \mathbb{E}[R_0 | \pi]$.

In order to maximize $J(\pi)$, action-value function $Q^\pi(s, a) = \mathbb{E}^\pi[R_t | s_t = s, a_t = a]$ is defined to measure the expectation of the discounted cumulative reward of choosing an action a in a state s under π . The action-value function is also called Q function [6]. But the true $Q^\pi(s, a)$ is unknown, so $Q(s, a)$ is used to estimate $Q^\pi(s, a)$ by temporal-difference (TD) learning. This process is based on Bellman operator T^π [6,25] as follows.

$$T^\pi Q(s, a) = r(s, a) + \mathbb{E}_{a' \sim \pi, s' \sim \rho^\pi} [\gamma Q(s', a')] \quad (1)$$

where s' is the successor state of s , $\rho^\pi = \int_{\mathcal{S}} \mathbb{E}_{a \sim \pi(s)} p(s' | s, a) ds$ and $p(s' | s, a)$ is the probability of transferring to s' with choosing a in s . T^π is a contraction mapping with unique fixed point $Q^\pi(s, a)$ [26]. Most RL methods use T^π to update $Q(s, a)$.

Actor-critic is a widely used RL architecture with two components, actor and critic [27,28]. Concretely, by the explored transitions and the Bellman operator, the critic estimates $Q(s, a)$. Then the actor updates π by maximizing $\mathbb{E}_{a \sim \pi} [Q(s, a)]$ using policy gradient ascent. After π is updated, it is used to explore new transitions.

2.2. TD3 method

TD3 is an efficient RL method with ER in actor-critic architecture. It is an improvement of a representative RL method, DDPG. Therefore, to present TD3 clearly, we first introduce DDPG.

DDPG concurrently trains a critic network Q_θ to estimate the Q function and an actor network π_ϕ to update the policy by ER which is used to exploit the experience of the transitions. For each training time step t , firstly, DDPG explores a new transition (s_t, a_t, r_t, s_{t+1}) by the learned policy and then stores this transition into a replay buffer $\mathcal{B}$. Secondly, DDPG randomly samples a batch of transitions from $\mathcal{B}$ for estimating the Q function by minimizing the following loss function in the critic network.

$$L(\theta) = \mathbb{E}_{(s, a, r, s') \sim \text{ER}(\mathcal{B})} [(Q_\theta(s, a) - z)^2] \quad (2)$$

$$z = r(s, a) + \gamma Q_{\theta'}(s', \pi_{\phi'}(s')) \quad (3)$$

where $(s, a, r, s') \sim \text{ER}(\mathcal{B})$ means that the batch of (s, a, r, s') is sampled from $\mathcal{B}$ by ER. $Q_{\theta'}$ and $\pi_{\phi'}$ are the corresponding target networks of Q_θ and π_ϕ . These two target networks can improve the stability of training Q_θ and π_ϕ . Thirdly, based on sampled transitions, DDPG updates the actor network to maximize the learned Q function:

$$\max_{\phi} \mathbb{E}_{s \sim \text{ER}(\mathcal{B})} [Q_\theta(s, \pi_\phi(s))] \quad (4)$$

Finally, DDPG updates two target networks by

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta' \quad (5)$$

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi' \quad (6)$$

where $0 < \tau < 1$.

TD3 is an important improvement of DDPG. It addresses the overestimation of the Q function in DDPG. TD3 trains two critic networks, Q_{θ_1} and Q_{θ_2} , to estimate the Q function. These networks are updated by minimizing the following loss functions.

$$L(\theta_i) = \mathbb{E}_{(s, a, r, s') \sim \text{ER}(\mathcal{B})} [(Q_{\theta_i}(s, a) - z)^2] \quad (7)$$

$$z = r(s, a) + \gamma \min_{i=1,2} Q_{\theta_i}(s', \pi_{\phi'}(s')) \quad (8)$$

TD3 delayed updates the policy to maximize $Q_{\theta_1}(s, \pi_\phi(s))$.

Like DDPG, TD3 uses ER to exploit the experience of the transitions. In this way, the last explored transitions are easy to be ignored. Thus the exploitation efficiency is limited in TD3.

2.3. PER: an improvement of ER

Prioritized experience replay (PER) assumes that a transition with the large estimation error of its Q function value should be frequently learned so as to reduce this error [9]. This type of error is called TD error and is defined as

$$|\delta_i| = |Q_\theta(s, a) - r(s, a) - \gamma Q_\theta(s', a')| \quad (9)$$

Based on the TD error, the probability of sampling a transition $(s, a, s', r)_i$ is as follows.

$$p(i) = \frac{\tilde{p}_i^\alpha}{\sum_j \tilde{p}_j^\alpha} \quad (10)$$

where $\tilde{p}_i = |\delta_i| + \epsilon$ is the priority of transition $(s, a, r, s')_i$ and ϵ is a small constant to ensure that every transition is sampled with non-zero probability. The exponent α is used to determine the priority level.

Furthermore, PER uses importance-sampling weights [29,30] to correct the bias introduced from sampling. The reason for this bias is that the distribution of the sampled states is different from the distribution of the states for the current Q function. The importance-sampling weight of each transition is

$$w(i) = \left(\frac{1}{N} \cdot \frac{1}{p(i)} \right)^\beta \quad (11)$$

where β is used to smooth out high variance importance-sampling weights and N is the replay buffer size.

PER can replay the transitions better than ER. However, it still has a problem as follows. The TD error of every transition is changed at each training time step, but PER only updates the priorities of the transitions replayed at the current training time step. In this way, the transitions with low priorities cannot be updated, leading to the problem that these transitions are ignored.

3. Limited exploitation efficiency

In this section, firstly, we analyze that if some explored transitions are ignored in training, the exploitation efficiency will be limited. Then, we present this problem in existing experience replay methods, e.g., ER and PER. Finally, we present how to improve the exploitation efficiency.

3.1. The exploitation efficiency limited by ignoring the transitions

Most RL methods update the Q function by exploiting the experience in the explored transitions. This update can be expressed as

$$Q_\theta(s, a) \doteq r(s, a) + \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}} [\gamma Q_\theta(s', \pi_\phi(a'))] \quad (12)$$

If all transitions in $\mathcal{B}$ are replayed sufficiently in the current training, the update of Q_θ will be the same in the Bellman operator for accurately estimating Q^{π_ϕ} . However, if some transitions are ignored, Q^{π_ϕ} cannot be estimated accurately by Q_θ . In this case, the exploitation efficiency will be limited. This view is detailed as follows.

In the training process, if the transitions in D_{p1} are ignored, the TD error of $(s, a, r, s') \in D_{p1}$ will not tend to 0 ($\delta_{(s,a,r,s')} \not\rightarrow 0$). Thus

$$\nabla_\theta L(\theta, D_{p1}) = \frac{1}{2} \mathbb{E}_{(s,a,r,s') \sim D_{p1}} [\delta_{(s,a,r,s')} \nabla_\theta Q_\theta(s, a)] \not\rightarrow 0 \quad (13)$$

Eq. (13) means that Q_θ cannot accurately estimate the Q function of the state-action pair corresponding to D_{p1} after the training.

3.2. The ignored transitions in existing experience replay

We present the ignored transitions in the existing experience replay methods, e.g., ER and PER. Figs. 1 and 2 illustrate this problem by presenting the expected replay number of each transition in $\mathcal{B}$ based on ER and PER, respectively. The darker the color of a

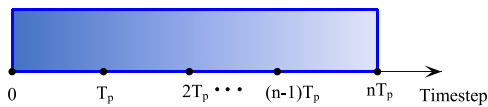


Fig. 1. Replaying the transitions by ER in replay buffer.

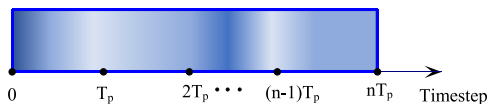


Fig. 2. Replaying the transitions by PER in replay buffer.

transition is, the higher the expected replay number of a transition is.

For the transition $(s, a, r, s')_i$ explored at training time step i , we denote the replay number of it until training time step l as rn_i^l . As we can see from Fig. 1, for $\forall j$ if $i < j < l$

$$\mathbb{E}[rn_i^l] < \mathbb{E}[rn_j^l] \quad (14)$$

The proof is as follows.

$$\mathbb{E}[rn_i^l] = \sum_{t=i+1}^l \frac{b}{|\mathcal{B}^t|} = \sum_{t=i+1}^l \frac{b}{t} \quad (15)$$

where b is the batch size and $|\mathcal{B}^t|$ is the number of transitions in $\mathcal{B}$ at training time step t . From (14) and Fig. 1, we find that the expected replay number of the currently explored transitions is much less than that of the past explored transitions. Therefore, the last explored transitions may be ignored.

As we can see from Fig. 2, some past transitions are ignored in PER. The reason is as follows. If the TD error of (s, a, r, s') tend to 0 at training time step i as

$$|\delta_{(s,a,r,s')}| = |Q_{\theta i}(s, a) - r(a, s) - \gamma Q_{\theta i}(s', a')| \rightarrow 0 \quad (16)$$

the priority of (s, a, r, s') will be small and thus (s, a, r, s') will be forgotten. If (s, a, r, s') has been forgotten in training time step j ($j > i$), it is hard to update the priority of (s, a, r, s') , because PER only updates the priorities of the transitions replayed at the current training time step. However, at training time step j , the TD error of (s, a, r, s') has already been changed due to $Q_{\theta j} \neq Q_{\theta i}$.

3.3. How to improve the exploitation efficiency

We give an example on two MuJoCo continuous control tasks, HalfCheetah and Ant [31,32], to present a solution for how to improve the exploitation efficiency. From the analysis in Section 3.1, we find that the exploitation efficiency can be improved by replaying all explored transitions in the current training. But this is a time consuming way because it will cause a large batch of the transitions to update the Q function and the policy in the current training. Therefore, we can cluster them and then replay the transitions sampled from all the clusters to ensure that all kinds of transitions are replayed sufficiently [33–35]. For illustrating this view, we construct a simple improvement of ER, SER, to cluster the transitions explored before 2×10^5 training time steps and sufficiently replay these kinds of transitions in subsequent training. The learning curves for HalfCheetah and Ant by TD3 with ER, PER, and SER are shown in Fig. 3. As we can see from Fig. 3, TD3_SER is more effective than TD3 and TD3_PER. This example illustrates that ignoring transitions will limit the exploitation efficiency for RL. It also illustrates that our analysis can provide an efficient way to improve the exploitation efficiency.

4. Methodology

In this section, firstly, we propose a clustering experience replay (CER) to improve the exploitation efficiency in the existing experience replay methods. Secondly, we construct TD3_CER to implement our CER on TD3. Then we theoretically analyze that our CER can sufficiently replay all kinds of transitions. Finally, we present the time and space complexities of CER, ER, and PER.

4.1. Clustering experience replay

Based on the analysis of Section 3, we propose CER in a divide-and-conquer framework based on time division. We divide the whole training process (from time step 0 to $T_{\max}$) into several periods with the length of T_p and divide $\mathcal{B}$ into different sub-buffers

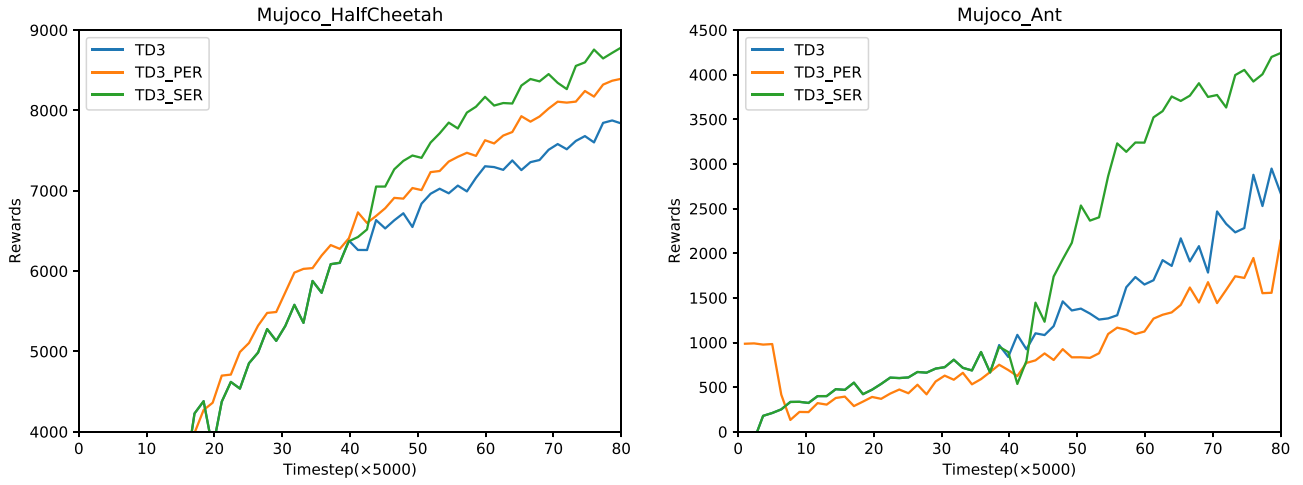


Fig. 3. Learning curves for the MuJoCo continuous action control tasks, HalfCheetah and Ant.

$\{\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_n\}$, where $n = \lceil \frac{T_{\max}}{T_p} \rceil$. We store $(s, a, r, s')_t$ explored at time step t into $\mathcal{B}_{\lceil \frac{t}{T_p} \rceil}$ and use k -means to cluster the transitions in $\mathcal{B}_j$ into k clusters at the end of period j (from time step $(j-1)T_p$ to jT_p , $j = 1, 2, \dots, n$). The number of the transitions of the m th cluster in $\mathcal{B}_j$ is denoted as $|\mathcal{B}_{jm}|$, where $m = 1, 2, \dots, k$. Here the transitions in one cluster can be seen as a kind of transition.

Based on the above clustering, we construct a conditional probability density function $p((s, a, r, s')|t)$ to effectively replay the different kinds of transitions. Firstly, we define the probability of replaying the transitions in $\mathcal{B}_j$ at time step t as

$$p(\mathcal{B}_j|t) = \begin{cases} \frac{1}{f(\lceil \frac{t}{T_p} \rceil)}, & j < \lceil \frac{t}{T_p} \rceil \\ 1 - \frac{\lceil \frac{t}{T_p} \rceil - 1}{f(\lceil \frac{t}{T_p} \rceil)}, & j = \lceil \frac{t}{T_p} \rceil \end{cases} \quad (17)$$

where f satisfies $f(\lceil \frac{t}{T_p} \rceil) > \lceil \frac{t}{T_p} \rceil$. Then, in order to ensure that the replay number of the transitions explored in a sub-buffer is the same as that of the transitions in another sub-buffer for the whole training process, we dynamically set the batch size b_j for the period j . For $\forall x, y$, if $1 \leq x \leq y \leq n$, b_j can ensure

$$\sum_{j=x}^n p(\mathcal{B}_x|jT_p)b_j = \sum_{j=y}^n p(\mathcal{B}_y|jT_p)b_j \quad (18)$$

How to compute b_j by (18) is shown in Appendix A.1. Finally, based on (17) and (18), we define the probability of replaying $(s, a, r, s')_i$ at time step t as

$$p((s, a, r, s')_i|t) = \begin{cases} p(\mathcal{B}_{\lceil \frac{t}{T_p} \rceil}|t) \cdot \frac{b_{\lceil \frac{t}{T_p} \rceil}}{|\mathcal{B}_{\lceil \frac{t}{T_p} \rceil}|}, & \lceil \frac{i}{T_p} \rceil = \lceil \frac{t}{T_p} \rceil \\ p(\mathcal{B}_{\lceil \frac{t}{T_p} \rceil}|t) \cdot \frac{b_{\lceil \frac{t}{T_p} \rceil}}{k|\mathcal{B}_{\lceil \frac{t}{T_p} \rceil}|_{m(i)}}, & \lceil \frac{i}{T_p} \rceil < \lceil \frac{t}{T_p} \rceil \end{cases} \quad (19)$$

where $b_{\lceil \frac{t}{T_p} \rceil}$ is the batch size of period $\lceil \frac{t}{T_p} \rceil$, $|\mathcal{B}_j|$ is the number of the transitions in $\mathcal{B}_j$ at time step t , and $m(i)$ is the cluster label of $(s, a, r, s')_i$. Eq. (19) can guarantee that the expected sum of the replayed number of any kind of transition is the same as that of another kind of transition in the whole training process (This conclusion is a theorem presented in Section 4.3).

In order to present our CER clearly, we summarize our CER in Algorithm 1. Besides, we give a diagram of it in Fig. 4. Compared with ER and PER in Figs. 1 and 2, our CER can ensure that every kind of transition will not be ignored.

Algorithm 1 CER.

- 1: Compute $c = \lceil \frac{t}{T_p} \rceil$
- 2: Store transition (s_t, a_t, r_t, s_{t+1}) into $\mathcal{B}_c$
- 3: Sample a batch of b_c transitions (s, a, r, s') according to (??)
- 4: **if** $\lceil \frac{t+1}{T_p} \rceil > c$ **then**
- 5: Use k -means to cluster the transitions in $\mathcal{B}_c$ into k clusters
- 6: **end if**
- 7: Return the sampled transitions

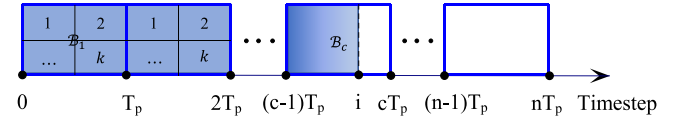


Fig. 4. Replaying the transitions by CER in replay buffer for time step i .

4.2. TD3_CER: TD3 based on clustering experience replay

We construct TD3_CER to implement our CER on TD3. There are six networks Q_{θ_1} , Q_{θ_2} , π_ϕ , Q'_{θ_1} , Q'_{θ_2} , and π'_ϕ in TD3_CER.

For each training time step t , firstly, TD3_CER explores a new transition by the policy π_ϕ and then stores this transition into sub-buffer $\mathcal{B}_{\lceil \frac{t}{T_p} \rceil}$. Secondly, TD3_CER samples a batch of $b_{\lceil \frac{t}{T_p} \rceil}$ transitions from $\mathcal{B}$ based on the conditional probability density function in (19). Q_{θ_1} and Q_{θ_2} for estimating Q^{π_ϕ} are updated by the sampled transitions to minimize the following loss function.

$$L(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \text{CER}(\mathcal{B})} [(Q_{\theta_i}(s, a) - r(s, a) - \gamma \min_{j=1,2} Q'_{\theta_j}(s', \pi_{\phi'}(s')))^2] \quad (20)$$

Finally, TD3_CER delayed updates π_ϕ by

$$\max_{\phi} \mathbb{E}_{s \sim \text{CER}(\mathcal{B})} [Q_{\theta_1}(s, \pi_\phi(s))] \quad (21)$$

and three target networks by

$$\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i \quad (22)$$

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi' \quad (23)$$

with d time steps, where $0 < \tau < 1$.

Furthermore, we have a trick to increase the diversity of the transitions [36]. The trick stores the transitions generated during not only the training process but also the testing process into the sub-buffer. The algorithm of TD3_CER is presented in Algorithm 2.

Algorithm 2 TD3_CER.

```

1: Initialize  $\theta_1, \theta_2, \phi, \theta'_1, \theta'_2$  and  $\phi'$ 
2: Divide replay buffer  $\mathcal{B}$  into  $\{\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_n\}$ 
3: for  $t = 0$  to  $T_{\max}$  do
4:   Select  $a_t = \pi_{\phi}(s_t) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma)$ 
5:   Execute  $a_t$ , then obtain  $r_t, s_{t+1}$ 
6:   Store transition  $(s_t, a_t, r_t, s_{t+1})$  and sample a batch of transitions  $(s, a, r, s')$  according to Algorithm 1
7:    $\pi_{\phi'}(s') = \pi_{\phi}(s') + \epsilon, \epsilon \sim \mathcal{N}(0, \tilde{\sigma})$ 
8:   Update  $\theta_i$  by minimizing (??)
9:   if  $t \bmod d$  then then
10:    Update  $\phi$  by solving (??)
11:    Update  $\theta'_i$  and  $\phi'$  by the (??) and (??)
12:   end if
13:   if  $t$  is the last time step in an episode then
14:    Test  $\pi_{\phi}$  in another episode and store the generated transitions in  $\mathcal{B}_c$ 
15:   end if
16: end for

```

4.3. Theoretical analysis

In this subsection, we theoretically analyze that the expected number of replaying any kind of transition is the same as that of replaying another kind of transition in the whole training process by CER. Thus every kind of transition can be replayed sufficiently.

Theorem 1. *If all kinds of transitions have the same expected replay number before clustering, the expected replay numbers of the different kinds of transitions will be the same by CER in the training process.*

Proof. We denote the replayed number of the transition (s, a, r, s') explored at time step i by CER as rn_i . In CER, the expectation of rn_i is

$$\begin{aligned} \mathbb{E}[rn_i] &= \sum_{t=i+1}^{cT_p} p(\mathcal{B}_c|t) \frac{b_c}{|\mathcal{B}_c^t|} + \sum_{t=cT_p+1}^{T_{\max}} p(\mathcal{B}_{\lceil \frac{t}{T_p} \rceil}|t) \frac{b_{\lceil \frac{t}{T_p} \rceil}}{k|\mathcal{B}_{c_m(i)}|} \\ &= \sum_{t=i+1}^{cT_p} p(\mathcal{B}_c|t) \frac{b_c}{|\mathcal{B}_c^t|} + \sum_{j=c+1}^n T_p p(\mathcal{B}_c|jT_p + 1) \frac{b_j}{k|\mathcal{B}_{c_m(i)}|} \end{aligned} \quad (24)$$

where $c = \lceil \frac{i}{T_p} \rceil$.

For $\mathcal{B}_c$, we denote the replay number of transitions in $\mathcal{B}_c$ as $S_{\mathcal{B}_c} = \sum_{(s,a,r,s') \in \mathcal{B}_c} rn_i$. The expectation of $S_{\mathcal{B}_c}$ by CER is

$$\begin{aligned} \mathbb{E}[S_{\mathcal{B}_c}] &= \sum_{i=(c-1)T_p}^{cT_p-1} \left[\sum_{t=i+1}^{cT_p} p(\mathcal{B}_c|t) \frac{b_c}{|\mathcal{B}_c^t|} + \sum_{j=c+1}^n T_p p(\mathcal{B}_c|jT_p + 1) \frac{b_j}{k|\mathcal{B}_{c_m(i)}|} \right] \\ &= p(\mathcal{B}_c|cT_p) b_c \sum_{i=(c-1)T_p}^{cT_p-1} \frac{1}{t - (c-1)T_p} + T_p \sum_{j=c+1}^n p(\mathcal{B}_c|jT_p) b_j \\ &= p(\mathcal{B}_c|cT_p) b_c T_p + T_p \sum_{j=c+1}^n p(\mathcal{B}_c|jT_p) b_j \\ &= T_p \sum_{j=c}^n p(\mathcal{B}_c|jT_p) b_j \end{aligned} \quad (25)$$

According to (18), for $\forall x, y \in \{1, 2, \dots, n\}$, we have

$$\mathbb{E}[S_{\mathcal{B}_x}] = \mathbb{E}[S_{\mathcal{B}_y}] \quad (26)$$

We denote the replay number of the m th kind of transitions in $\mathcal{B}_c$ as $S_{\mathcal{B}_{c_m}} = \sum_{(s,a,r,s') \in \mathcal{B}_{c_m}} rn_i$. The expectation of $S_{\mathcal{B}_{c_m}}$ by CER is

Table 1

Time and space complexities.

	ER	PER	CER
Time	$O(1)$	$O(\log(N))$	$O(k \cdot \tilde{N} \cdot d)$
Space	$O(N \cdot d)$	$O(N \cdot d)$	$O(N \cdot d)$

$$\begin{aligned} \mathbb{E}[S_{\mathcal{B}_{c_m}}] &= \sum_{(s,a,r,s') \in \mathcal{B}_{c_m}} \left[\sum_{t=i+1}^{cT_p} p(\mathcal{B}_c|t) \frac{b_c}{|\mathcal{B}_c^t|} + \sum_{j=c+1}^n T_p p(\mathcal{B}_c|jT_p + 1) \frac{b_j}{k|\mathcal{B}_{c_m}|} \right] \\ &= \sum_{(s,a,r,s') \in \mathcal{B}_{c_m}} \sum_{t=i+1}^{cT_p} p(\mathcal{B}_c|t) \frac{b_c}{|\mathcal{B}_c^t|} + \frac{T_p}{k} \sum_{j=c+1}^n p(\mathcal{B}_c|jT_p) b_j \end{aligned} \quad (27)$$

According to the assumption in Theorem 1, for $\forall m_1, m_2 \in \{1, 2, \dots, k\}$, we have

$$\sum_{(s,a,r,s') \in \mathcal{B}_{c_{m_1}}} \sum_{t=i+1}^{cT_p} p(\mathcal{B}_c|t) \frac{b_c}{|\mathcal{B}_c^t|} = \sum_{(s,a,r,s') \in \mathcal{B}_{c_{m_2}}} \sum_{t=i+1}^{cT_p} p(\mathcal{B}_c|t) \frac{b_c}{|\mathcal{B}_c^t|} \quad (28)$$

Therefore,

$$\mathbb{E}[S_{\mathcal{B}_{c_{m_1}}}] = \mathbb{E}[S_{\mathcal{B}_{c_{m_2}}}] \quad (29)$$

According to $\mathbb{E}[S_{\mathcal{B}_c}] = \sum_{m=1}^k \mathbb{E}[S_{\mathcal{B}_{c_m}}]$ and (26), for $\forall x, y \in \{1, 2, \dots, n\}$ and $\forall m_1, m_2 \in \{1, 2, \dots, k\}$, we have

$$\mathbb{E}[S_{\mathcal{B}_{x_{m_1}}}] = \mathbb{E}[S_{\mathcal{B}_{y_{m_2}}}] \quad (30)$$

□

As we can see from Theorem 1, CER can sufficiently replay all kinds of transitions.

4.4. Time and space complexities

We present the time and space complexities of CER and the existing experience replay methods in Table 1, where N is the size of replay buffer $\mathcal{B}$, $\tilde{N}$ is the size of sub-buffer $\mathcal{B}_i$ ($\tilde{N} \ll N$), and d is the dimension of a transition. For time complexity, CER only clusters the transitions in the sub-buffer rather than the transitions in the whole replay buffer. Thus the time complexity is reduced from $O(k \cdot N \cdot d)$ to $O(k \cdot \tilde{N} \cdot d)$. For space complexity, the space requirement of CER is no more than that of the existing experience replay methods.

5. Experiments

In this section, firstly, we compare our TD3_CER with four RL methods to illustrate the effectiveness of TD3_CER. Then we analyze the parameter sensitivity of the batch size b and the cluster number k for TD3_CER to illustrate that our TD3_CER is not sensitive to them. Finally, we give a running time comparison.

5.1. Experiment result

In this subsection, we evaluate our new algorithm TD3_CER on the MuJoCo continuous control tasks, which are interfaced through OpenAI Gym [31,32]. The environments tested in this paper are shown in Fig. 5. We compare our TD3_CER against DDPG [14], TD3 [15], SAC [37], and TD3_PER [9].

The parameters setting is shown in Appendix A.2. All of our experiments are run on one device with a CPU of Intel core i7-8700K and a GPU of NVIDIA GTX1080Ti. For all tested methods in this paper, each task runs 10^6 time steps, and evaluation is performed every 5000 time steps. Each evaluation reports the average cumulative reward over 10 episodes. Our experimental results are based on 10 random seeds of the Gym simulator and the network initialization.

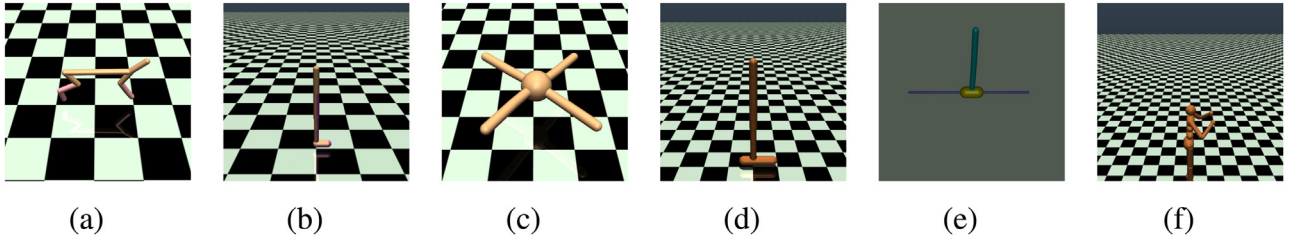


Fig. 5. The MuJoCo environments in our experiment (a) HalfCheetah, (b) Walker2d, (c) Ant, (d) Hopper, (e) InvDoublePendulum, (f) Humanoid.

Table 2

Average max cumulative reward over 10 trails of 1 million time steps. Maximum value for each task is bolded. $\pm$ corresponds to the standard deviation over trials.

Environment	TD3_CER	TD3_PER	TD3	SAC	DDPG
HalfCheetah	12350.57 $\pm$ 629.71	10330.37 $\pm$ 827.45	10093.65 $\pm$ 571.46	8311.89 $\pm$ 2530.08	8856.27 $\pm$ 1374.17
Walker2d	5298.36 $\pm$ 530.36	4928.43 $\pm$ 1755.66	4296.81 $\pm$ 484.35	4384.79 $\pm$ 399.29	4016.18 $\pm$ 753.55
Ant	5769.94 $\pm$ 325.84	4891.27 $\pm$ 563.57	5241.53 $\pm$ 402.14	4643.07 $\pm$ 742.58	2364.18 $\pm$ 583.42
Hopper	3697.72 $\pm$ 96.34	3404.83 $\pm$ 1183.76	3453.59 $\pm$ 166.03	3498.18 $\pm$ 125.67	3633.78 $\pm$ 110.72
Humanoid	5466.49 $\pm$ 352.49	5271.88 $\pm$ 128.50	4946.29 $\pm$ 659.84	5412.55 $\pm$ 194.23	691.62 $\pm$ 488.43
InvDoublePendulum	9359.03 $\pm$ 1.66	9359.76 $\pm$ 0.20	9359.64 $\pm$ 0.34	9359.78 $\pm$ 0.35	9359.34 $\pm$ 0.57

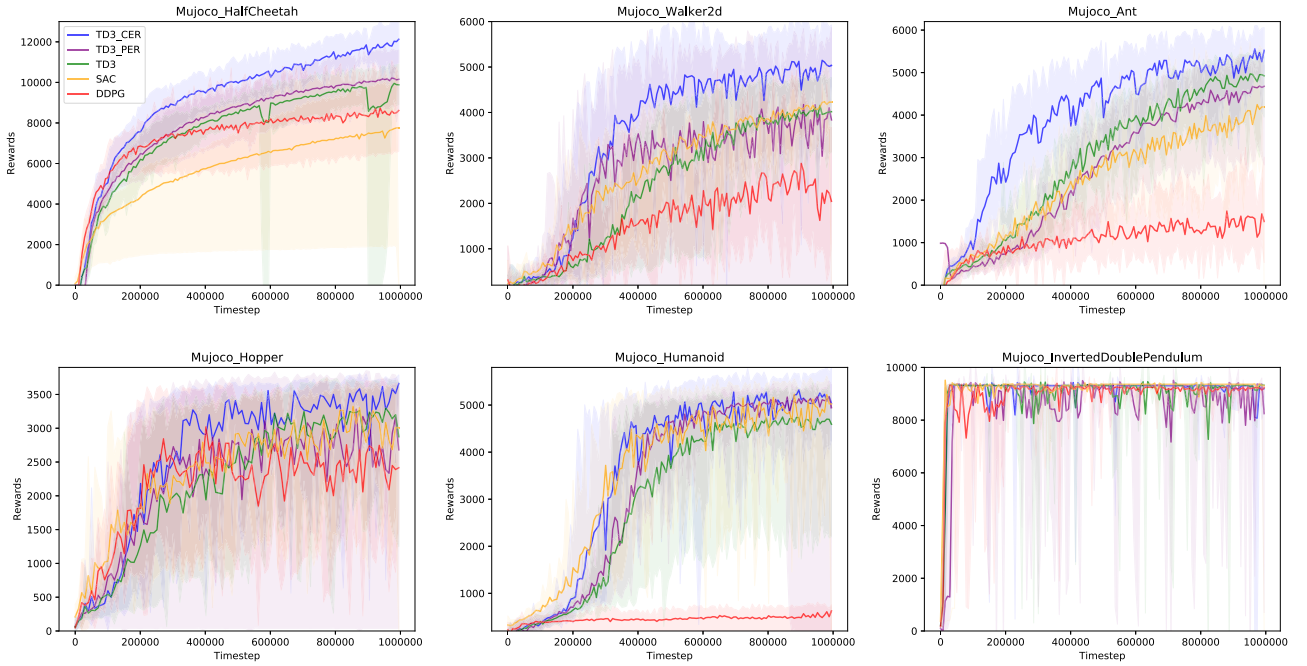


Fig. 6. Learning curves for the MuJoCo continuous action control tasks. The curves in this figure are smoothed uniformly for visual clarity.

The numerical results and the learning curves of our experiment are presented in Table 2 and Fig. 6, respectively. As we can see from Table 2 and Fig. 6, the max average cumulative reward of our TD3_CER is higher than that of other compared methods and CER can make the agent learn a good policy more quickly than ER and PER, especially for HalfCheetah, Walker2d, and Ant.

5.2. Parameter sensitivity analysis

In this subsection, we analyze the parameter sensitivity of the batch size b_j and the cluster number k for TD3_CER on HalfCheetah and Ant. For batch size, we test $\bar{b} = 128, 256, 512$ ($\bar{b}$ is the average of b_1 and $b_{\max}$), respectively. For cluster number, we test $k = 2, 3, 4$, respectively. The learning curves are presented in Figs. 7 and 8. As we can see from Figs. 7 and 8, the learning curves in each figure are similar when $\bar{b}$ and k are changed. Thus TD3_CER is not sensitive to $\bar{b}$ and k .

We set the batch size according to Appendix A.1 to ensure that the expected replay numbers of the different kinds of transitions will be the same. Thus TD3_CER will not be sensitive to $\bar{b}$ as long as the $\bar{b}$ is in a reasonable range.

In the divide-and-conquer framework, we put the transitions explored in a time period into the corresponding sub-buffer. We cluster the transitions in a sub-buffer for Halfcheetah and Ant, and then visualize the clustering results in Fig. 9. From the distribution of transitions in Fig. 9, the transitions in a sub-buffer are roughly divided into 2 to 3 clusters. Therefore, whether k is 2, 3, or 4, various kinds of samples can be roughly separated. Thus TD3_CER is not sensitive to k .

5.3. Running time comparison

We compare the running time of ER, PER, and CER in TD3. The comparison results are in Table 3.

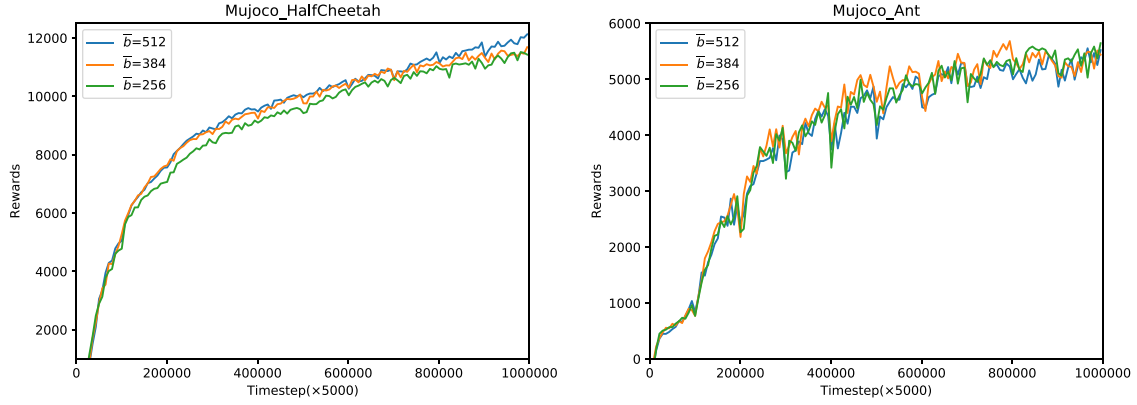


Fig. 7. Parameter sensitivity of $\bar{d}$ for TD3_CER. Learning curves for the MuJoCo continuous action control tasks, HalfCheetah and Ant.

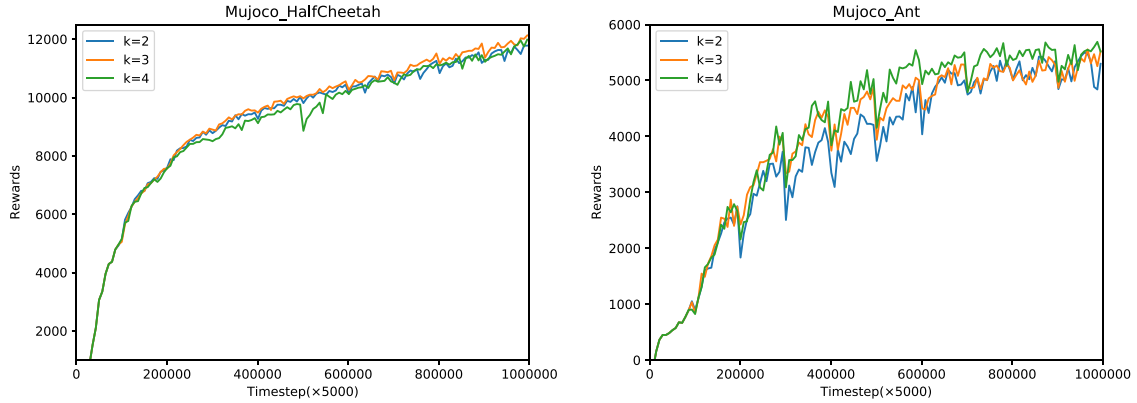


Fig. 8. Parameter sensitivity of k for TD3_CER. Learning curves for the MuJoCo continuous action control tasks, HalfCheetah and Ant.

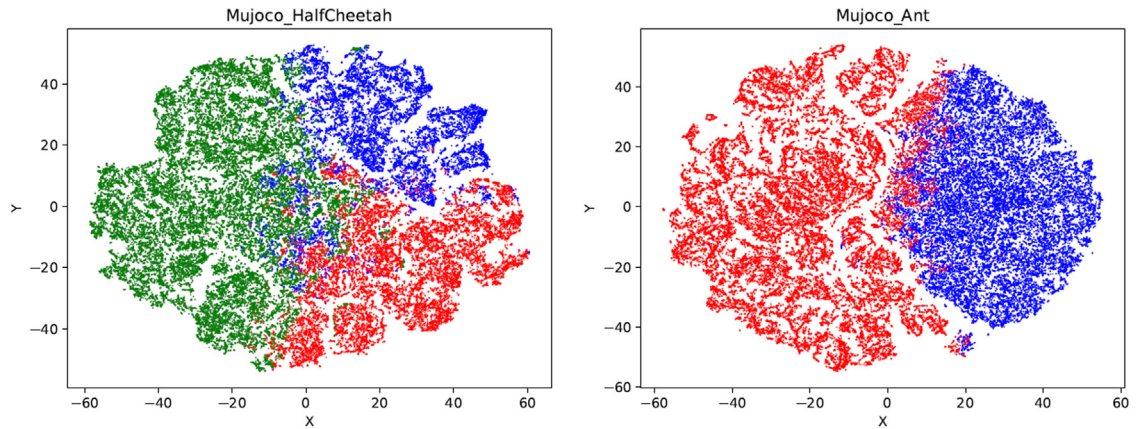


Fig. 9. Clustering results of transitions in a sub-buffer for Halfcheetah and Ant.

Table 3

Running time ($\times 10^3$).

	ER	PER	CER	dimension
HalfCheetah	7.84	8.25	8.47	41
Walker2d	7.35	8.66	8.60	41
Ant	8.66	9.70	10.75	231
Hopper	7.33	8.01	7.93	26
Humanoid	12.19	13.39	14.98	770
InvDoublePendulum	7.69	8.24	7.84	24

As we can see from Table 3, there is little difference in running time between CER and the existing experience replay methods. When d is small, such as in Hopper and InvDoublePendulum,

CER requires negligible extra running time. When d is large, such as in Ant and Humanoid, CER requires little more time to finish the same steps of training than the existing experience replay methods. As we can see from Fig. 6, after running about 4×10^5 time steps, CER can achieve the effect that other methods achieve until 1 million time steps. Therefore, although CER requires little more time to finish the same steps of training than the existing experience methods when d is large, it still can achieve effective exploration in RL.

6. Conclusion

In RL, existing experience replay methods are easy to ignore some explored samples (transitions), leading to the low exploita-

tion efficiency. In this paper, we propose CER to improve the exploitation efficiency for RL. The motivation is that the experience hidden in all explored samples should be sufficiently mined in the current training. In order to avoid high time consumption caused by replaying all of the explored samples, CER clusters them and then replays the sampled ones from all the clusters, which is in a divide-and-conquer framework based on time division.

There are three main contributions in this paper. Firstly, we analyze the limitation of the exploitation efficiency in existing reinforcement learning methods in detail. This analysis not only clarifies the motivation of our method but also provides a reference for how to improve the exploration efficiency of reinforcement learning, i.e., considering the diversity of samples. Secondly, we combine clustering into experience replay methods to mine the experience hidden in all kinds of samples. In this way, we not only achieve our motivation with a low time consumption, but also avoid the uneven replay of various kinds of the explored samples caused by the uneven sample distribution from the perspective of the similarity between the explored samples. Finally, we prove the effectiveness of CER through experiments and discover a group of suitable parameters for CER.

Our method also has a limitation. CER uses a well-known clustering method, k -means, to cluster the explored samples because k -means has low time consumption. But k -means is not suitable for high-dimensional data [38]. Therefore, the exploration efficiency of CER is limited when the explored samples live in a high-dimensional space. A useful tool for this issue is the deep clustering method where the mapping from the original data space to a low-dimensional space is learned by the deep neural network [39]. It is promising that the exploitation efficiency can be further improved by developing a suitable deep clustering method for the data composed of the explored samples. This can be a topic of our future work.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by The National Nature Science Foundation of China under Grant no. 61772120.

Appendix A. My appendix

A1. Batch size in CER

We denote time steps from $(j-1)T_p$ to jT_p as peroid j . Based on (17), for each peroid j , $1 \leq j \leq n$, we present the probability of replaying transitions in the sub-buffers and batch size b_j in Table A.4. Here we assume $b_j < b_{\max}$.

Table A1

The probability of replaying transitions in the sub-buffers and batch size b_j .

	Period 1	Period 2	Period 3	...	Period $c-1$	Period c
B_1	$1 - \frac{0}{f(1)}/b_1$	$\frac{1}{f(2)}/b_2$	$\frac{1}{f(3)}/b_3$	...	$\frac{1}{f(c-1)}/b_{c-1}$	$\frac{1}{f(c)}/b_c$
B_2		$1 - \frac{1}{f(2)}/b_2$	$\frac{1}{f(3)}/b_3$	...	$\frac{1}{f(c-1)}/b_{c-1}$	$\frac{1}{f(c)}/b_c$
B_3			$1 - \frac{2}{f(3)}/b_3$	...	$\frac{1}{f(c-1)}/b_{c-1}$	$\frac{1}{f(c)}/b_c$
$\vdots$			$\vdots$	$\vdots$	$\vdots$	$\vdots$
B_{n-1}					$1 - \frac{n-2}{f(n-1)}/b_{n-1}$	$\frac{1}{f(n)}/b_n$
B_n						$1 - \frac{1}{f(n)}/b_n$

Table A2

Parameter setting.

Parameter	Value	Brief explanation
T_p	10^5	Length of period
k	3	The number of clusters for each period
$f(c)$	$f(c) = 2c + 6$	$f(c)$ in (17)
b_c	$b_c = \min(256 \cdot \frac{c+3}{4}, 768)$	batch size
γ	0.99	Discount factor
τ	0.005	Target network update rate
σ	0.1	Std of noise added to policy
$\tilde{\sigma}$	0.2	Std of noise added to target policy
e	2	Frequency of delayed policy updates
α	0.0003	Learning rate of actor and critic networks

According to Table A.4 and (18),

$$\left(1 - \frac{c-2}{f(c-1)}\right)b_{c-1} + \sum_{i=c}^n \frac{1}{f(i)}b_i = \left(1 - \frac{c-1}{f(c)}\right)b_c + \sum_{i=c+1}^n \frac{1}{f(i)}b_i \quad (\text{A.1})$$

After simplification, we have

$$\frac{b_c}{b_{c-1}} = \frac{(f(c-1) - c + 2)f(c)}{f(c-1)(f(c) - c)} \quad (\text{A.2})$$

In experiments, we set $f(c) = 2c + 6$, according to (A.2), we have

$$\frac{b_c}{b_{c-1}} = \frac{c+3}{c+2} \Rightarrow \frac{b_c}{b_1} = \frac{c+3}{4} \quad (\text{A.3})$$

A2. Parameter setting

The parameters setting is shown in Table A.5.

References

- [1] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J.A. Ojea, E. Solowjow, S. Levine, Residual reinforcement learning for robot control, in: 2019 International Conference on Robotics and Automation (ICRA), IEEE, Montreal, Canada, 2019, pp. 6023–6029.
- [2] J. Kober, J.A. Bagnell, J. Peters, Reinforcement learning in robotics: a survey, *Int. J. Robot. Res.* 32 (11) (2013) 1238–1274.
- [3] A. Folkers, M. Rick, C. Büskens, Controlling an autonomous vehicle with deep reinforcement learning, in: 2019 IEEE Intelligent Vehicles Symposium (IV), IEEE, Paris France, 2019, pp. 2025–2031.
- [4] B.R. Kiran, I. Sobh, V. Talpaert, P. M. Annon, A.A.A. Sallab, S. Yogamani, P. Pérez, Deep reinforcement learning for autonomous driving: a survey, *arXiv preprint arXiv:2002.00444* (2020).
- [5] K.P. Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012.
- [6] R.S. Sutton, A.G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [7] W. Sihang, W. Jiapeng, M. Weihong, J. Lianwen, Precise detection of chinese characters in historical documents with deep reinforcement learning, *Pattern Recog nit.* 107 (2020) 107503.
- [8] L.P. Kaelbling, M.L. Littman, A.W. Moore, Reinforcement learning: a survey, *J. Artif. Intell. Res.* 4 (1996) 237–285.
- [9] T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized experience replay, in: International Conference on Learning Representations, San Juan, Puerto Rico, 2016.
- [10] S. Adam, L. Busoniu, R. Babuska, Experience replay for real-time reinforcement learning control, *IEEE Trans. Syst., Man, Cybern., Part C (Appl. Rev.)* 42 (2) (2011) 201–212.
- [11] Z. Wang, V. Bapst, N. Heess, V. Mnih, R. Munos, K. Kavukcuoglu, N. de Freitas, Sample efficient actor-critic with experience replay, in: International Conference on Learning Representations, Toulon, France, 2017.
- [12] S. Fujimoto, D. Meger, D. Precup, An equivalence between loss functions and non-uniform sampling in experience replay, in: Conference on Neural Information Processing Systems, vol. 33, 2020, pp. 142 19–14230.
- [13] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.
- [14] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, in: International Conference on Learning Representations, San Juan, Puerto Rico, 2016.
- [15] S. Fujimoto, H. Van Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: International Conference on Machine Learning, Stockholm, Sweden, vol. 80, 2018, pp. 1 587–1596.

- [16] M. Li, T. Huang, W. Zhu, Adaptive exploration policy for exploration–exploitation tradeoff in continuous action control optimization, *Int. J. Mach. Learn. Cybern.* 12 (12) (2021) 3491–3510.
- [17] C. Gehring, D. Precup, Smart exploration in reinforcement learning using absolute temporal difference errors, in: *Proceedings of the 2013 International Conference on Autonomous Agents and Multi-Agent Systems*, Paul, MN, USA, 2013, pp. 1037–1044.
- [18] M. Zhang, Weighted clustering ensemble: a review, *Pattern Recognit.* 124 (2021) 108428.
- [19] A. Saxena, M. Prasad, A. Gupta, N. Bharill, O.P. Patel, A. Tiwari, M.J. Er, W. Ding, C.-T. Lin, A review of clustering techniques and developments, *Neurocomputing* 267 (2017) 664–681.
- [20] J. MacQueen, Some methods for classification and analysis of multivariate observations, in: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, Oakland, CA, USA, vol. 1, 1967, pp. 281–297.
- [21] P. Fränti, S. Sieranoja, How much can k-means be improved by using better initialization and repeats?, *Pattern Recognit.* 93 (2019) 95–112.
- [22] M. Jabi, M. Pedersoli, A. Mitiche, I.B. Ayed, Deep clustering: on the link between discriminative models and k-means, *IEEE Trans. Pattern Anal. Mach. Intell.* 43 (6) (2019) 1887–1896.
- [23] D.P. Bertsekas, J.N. Tsitsiklis, Neuro-dynamic programming: an overview, in: *Proceedings of 1995 34th IEEE Conference on Decision and Control*, vol. 1, IEEE, 1995, pp. 560–564.
- [24] W.B. Powell, *Approximate Dynamic Programming: Solving the Curses of Dimensionality*, vol. 703, John Wiley & Sons, 2007.
- [25] Z. Song, R. Parr, L. Carin, Revisiting the softmax bellman operator: new benefits and new perspective, in: *International Conference on Machine Learning*, vol. 97, PMLR, California, USA, 2019, pp. 5916–5925.
- [26] F.S. Melo, Convergence of Q-Learning: A Simple Proof, *Tech. Rep.*, 2001.
- [27] V.R. Konda, J.N. Tsitsiklis, Actor-critic algorithms, in: *Advances in Neural Information Processing Systems*, Denver, CO, USA, 2000, pp. 1008–1014.
- [28] I. Grondman, L. Busoniu, G.A. Lopes, R. Babuska, A survey of actor-critic reinforcement learning: standard and natural policy gradients, *IEEE Trans. Syst., Man, Cybern., Part C (Appl. Rev.)* 42 (6) (2012) 1291–1307.
- [29] S. Han, Y. Sung, Dimension-wise importance sampling weight clipping for sample-efficient reinforcement learning, in: *International Conference on Machine Learning*, PMLR, CA, USA, 2019, pp. 2586–2595.
- [30] A.R. Mahmood, H. Van Hasselt, R.S. Sutton, Weighted importance sampling for off-policy learning with linear function approximation, in: *Conference on Neural Information Processing Systems*, Montreal, Quebec, Canada, 2014, pp. 3014–3022.
- [31] E. Todorov, T. Erez, Y. Tassa, Mujoco: a physics engine for model-based control, in: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, Algarve, Portugal, 2012, pp. 5026–5033.
- [32] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, *Openai gym*, arXiv preprint arXiv:1606.01540(2016).
- [33] R. Xu, D. Wunsch, *Clustering*, vol. 10, John Wiley & Sons, 2008.
- [34] R. Li, X. Yang, X. Qin, W. Zhu, Local gap density for clustering high-dimensional data with varying densities, *Knowledge-Based Syst.* 184 (2019) 104905.
- [35] T. Huang, S. Wang, W. Zhu, An adaptive kernelized rank-order distance for clustering non-spherical data with high noise, *Int. J. Mach. Learn. Cybern.* 11 (2020) 1735–1747.
- [36] S. Khadka, S. Majumdar, T. Nassar, Z. Dwiell, E. Tumer, S. Miret, Y. Liu, K. Tumer, Collaborative evolutionary reinforcement learning, in: *International Conference on Machine Learning*, PMLR, Long Beach, CA, USA, 2019, pp. 3341–3350.
- [37] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: *International Conference on Machine Learning*, Stockholm, Sweden, vol. 80, 2018, pp. 1861–1870.
- [38] Y. Yang, J. Cai, H. Yang, Y. Li, X. Zhao, ISBFK-means: a new clustering algorithm based on influence space, *Expert Syst. Appl.* 201 (2022) 117018.
- [39] L. Zhou, B. Xiao, X. Liu, J. Zhou, E.R. Hancock, et al., Latent distribution preserving deep subspace clustering, in: *28th International Joint Conference on Artificial Intelligence*, York, 2019, pp. 4440–4446.

Min Li is currently pursuing the doctor's degree with the Institute of Fundamental and Frontier Sciences, University of Electronic Science and Technology of China, Chengdu, China. Her current research interests are deep learning and reinforcement learning.

Tianyi Huang is currently pursuing the doctor's degree with the Institute of Fundamental and Frontier Sciences, University of Electronic Science and Technology of China, Chengdu, China. His current research interests are clustering and reinforcement learning.

William Zhu received the M.S. degree in mathematics from Xinjiang University, Wulumuqi, Xinjiang, China, in 1986, the M.S. degree in systems engineering from the University of Arizona, Tucson, AZ, USA, in 2002, and the Ph.D. degree from the University of Auckland, Auckland, New Zealand, in 2007. He is currently a Professor at the University of Electronic Science and Technology of China, Chengdu, China. His research interests include granular computing, rough set, data mining, and artificial intelligence.